

**INSTITUTO
FEDERAL**

Santa Catarina

Redes Industriais

Professor

Heron Eduardo de Lima Ávila, Dr.

heron.avila@ifsc.edu.br

Redes Industriais

Tópico de aula

**Comunicação com Arduino e Eclipse E3 Studio
via MODBUS RTU**

Professor

Heron Eduardo de Lima Ávila, Dr.

heron.avila@ifsc.edu.br

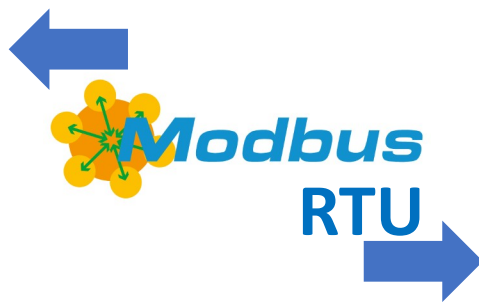
Rede a ser configurada

➤ Conexão dos dispositivos via MODBUS RTU

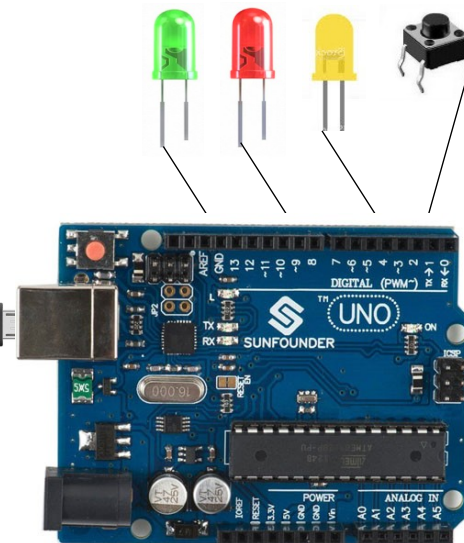
MESTRE



TAGs de
comunicação



ESCRAVO

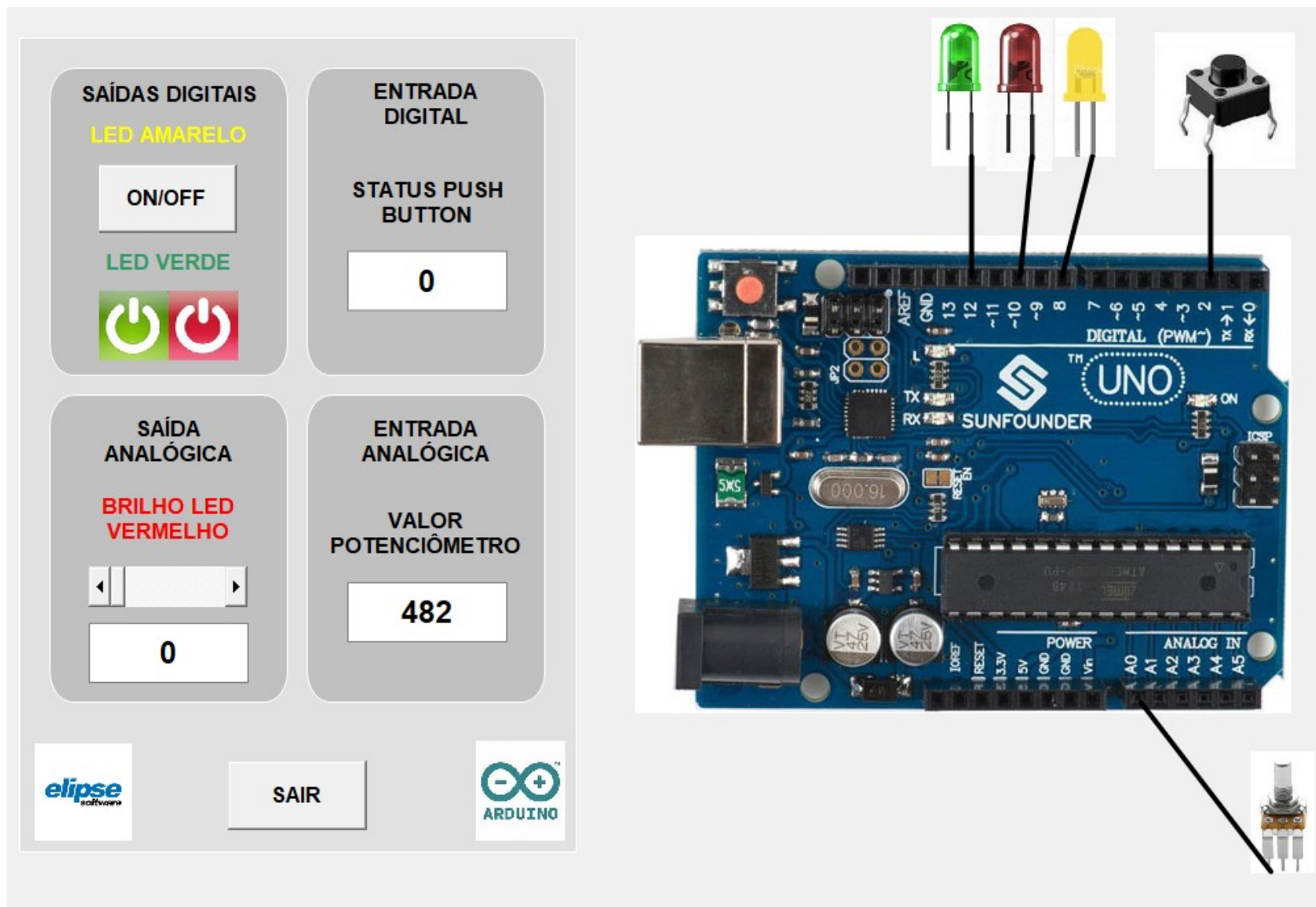


Registradores
internos

Pinos e TAGs
dos pinos

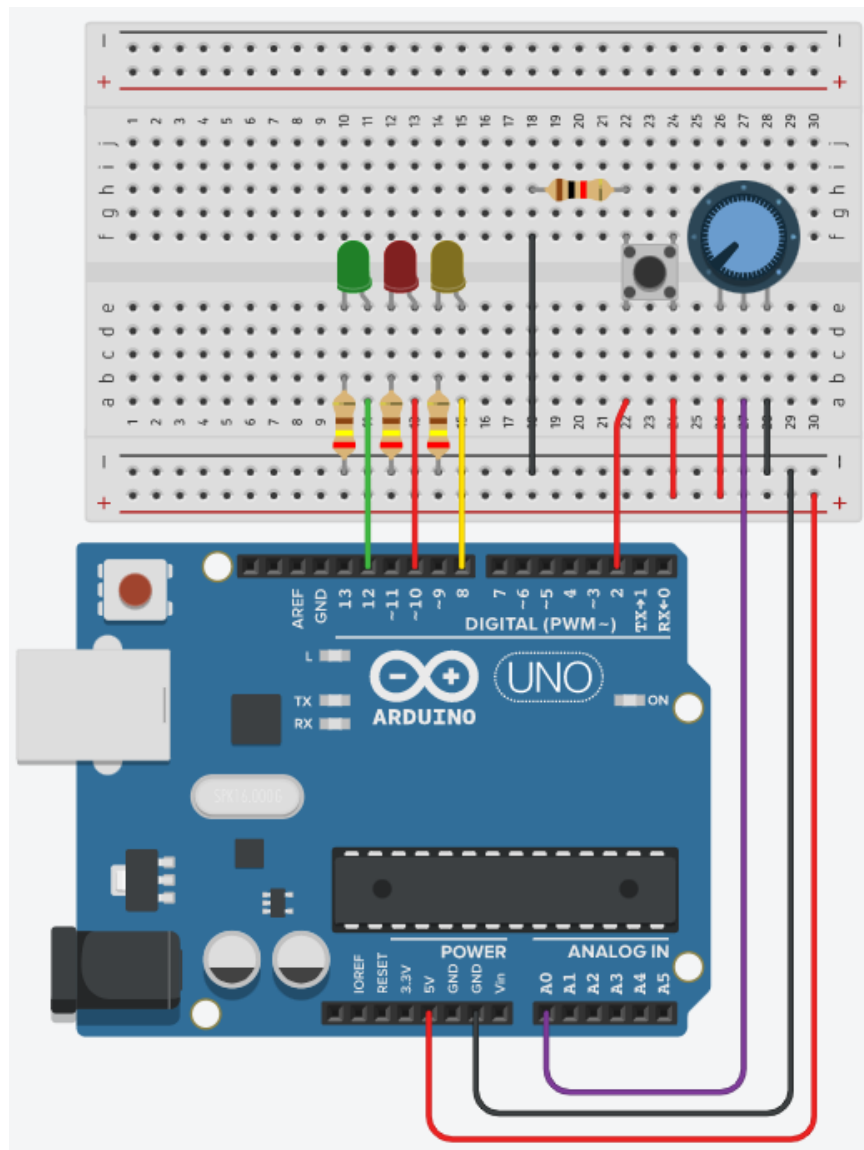
Rede a ser configurada

➤ Interface do Supervisor no E3 Studio



Rede a ser configurada

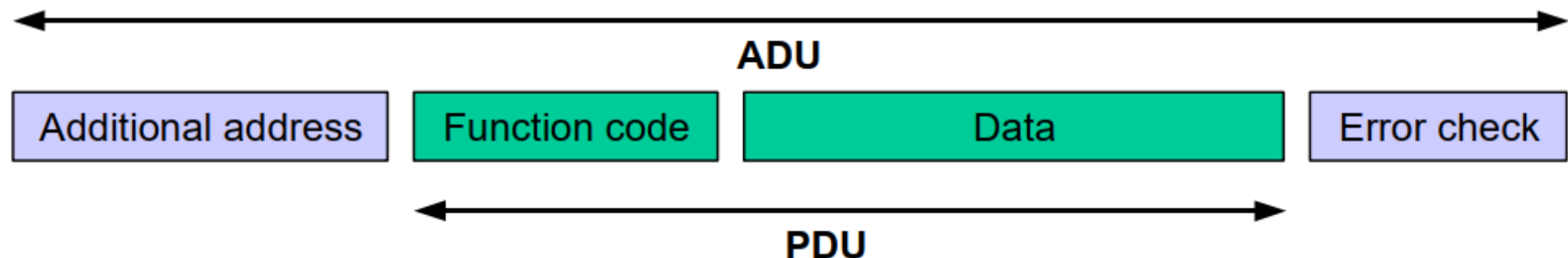
➤ Detalhes da montagem do circuito



Comunicação via MODBUS (relembrando)

➤ Modelo de dados MODBUS

- Quadro MODBUS geral:



- O protocolo MODBUS define uma unidade de dados de protocolo (*Protocolo Data Unit - PDU*) independente das camadas de comunicação subjacentes.
- O mapeamento do protocolo MODBUS em barramentos ou rede específicos pode introduzir alguns campos adicionais na unidade de dados de aplicação (*Application Data Unit - ADU*).
- A ADU é construída pelo mestre que inicia uma transação MODBUS. O código de função indica ao escravo que tipo de ação a ser realizada.

Comunicação via MODBUS (relembrando)

➤ Modelo de dados MODBUS

- O MODBUS baseia seu modelo de dados em uma série de tabelas que apresentam funções distintas para troca de mensagens (leitura/escrita de dados e *broadcast*).
- As quatro tabelas principais são:

Primary tables	Object type	Type of	Comments
Discretes Input	Single bit	Read-Only	This type of data can be provided by an I/O system.
Coils	Single bit	Read-Write	This type of data can be alterable by an application program.
Input Registers	16-bit word	Read-Only	This type of data can be provided by an I/O system
Holding Registers	16-bit word	Read-Write	This type of data can be alterable by an application program.

- **Dados de 1 bit:**
 - Entradas (inputs): leitura do escravo;
 - Bobinas (coils): podem ser lidos ou escritos no escravo.
- **Dados de 16 bits (registros):**
 - Entradas (inputs): leitura do escravo;
 - Retentivos (holding): podem ser lidos ou escritos no escravo.

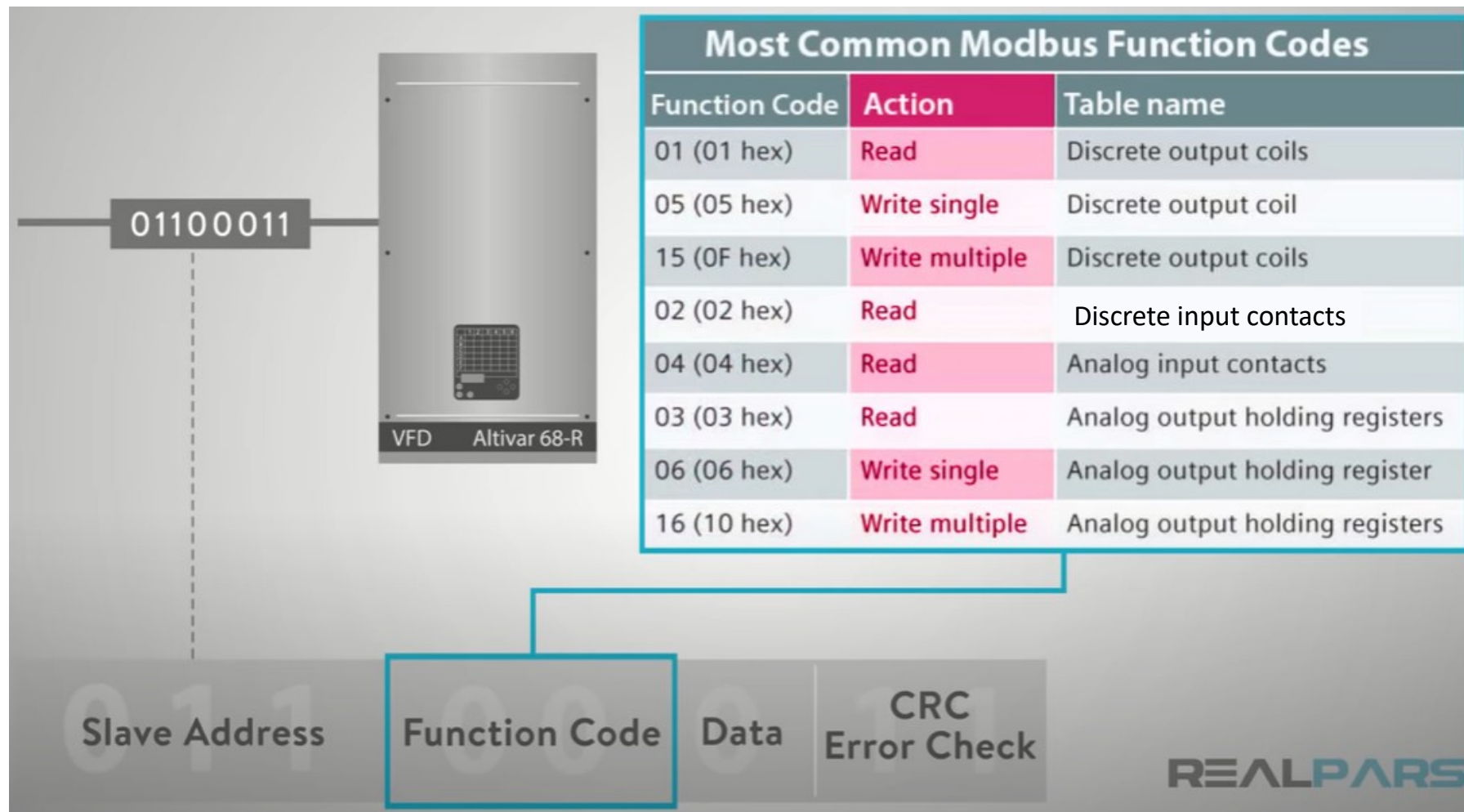
Comunicação via MODBUS (relembrando)

➤ Modelo de dados MODBUS

- Endereços lógicos dos dados (memória é dividida em registradores de 16 bits):
 - **00001 a 09999** – *coils*: solenóides, saídas discretas para os atuadores ON-OFF, utilizam um bit. Cada registrador comporta 16 saídas;
 - **10001 a 19999** – *inputs*: entradas discretas para os sensores ON-OFF, utilizam um bit. Cada registrador comporta 16 entradas;
 - **30001 a 39999** – *inputs registers*: entradas analógicas, utilizam registradores de 16 bits para os valores obtidos dos conversores A/D a partir dos sinais dos sensores analógicos;
 - **40001 a 49999** – *holding registers*: registradores de memórias com 16 bits para os valores utilizados internamente nos CLP's;
- Na prática todos os endereços lógicos variam de 0 a 9998 e a identificação está associada ao tipo do serviço (código da função).

Comunicação via MODBUS (relembrando)

- Códigos de função de leitura/escrita mais comuns

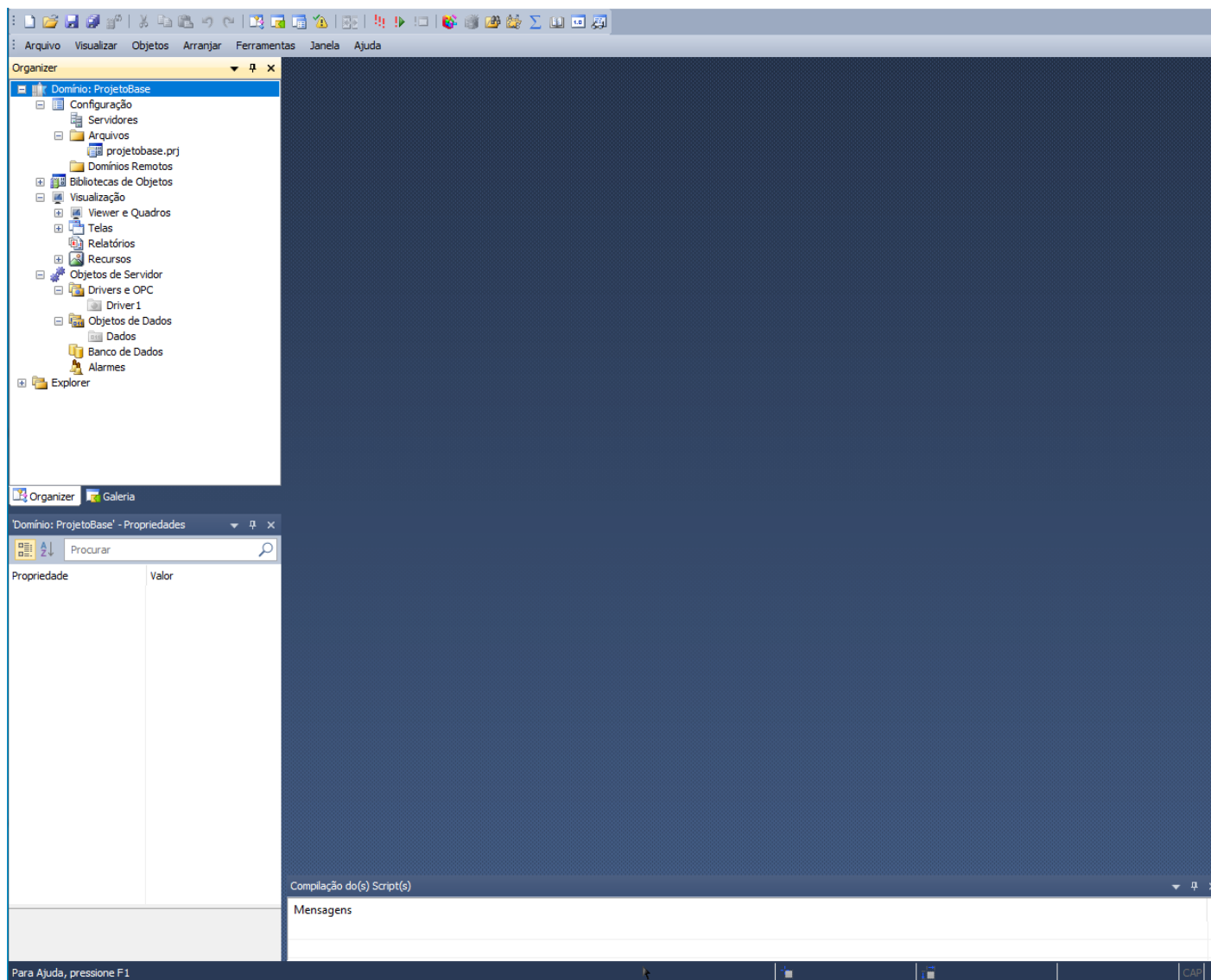


➤ O que é o Eclipse E3 Studio

- Ferramenta única de configuração do sistema, servindo como plataforma universal de desenvolvimento. Possui um ambiente moderno e amigável, incluindo desde a configuração da comunicação até editores de scripts e de gráficos para a criação das telas de operação. Permite que um mesmo aplicativo seja editado por diversos usuários simultaneamente, facilitando o trabalho em equipe.

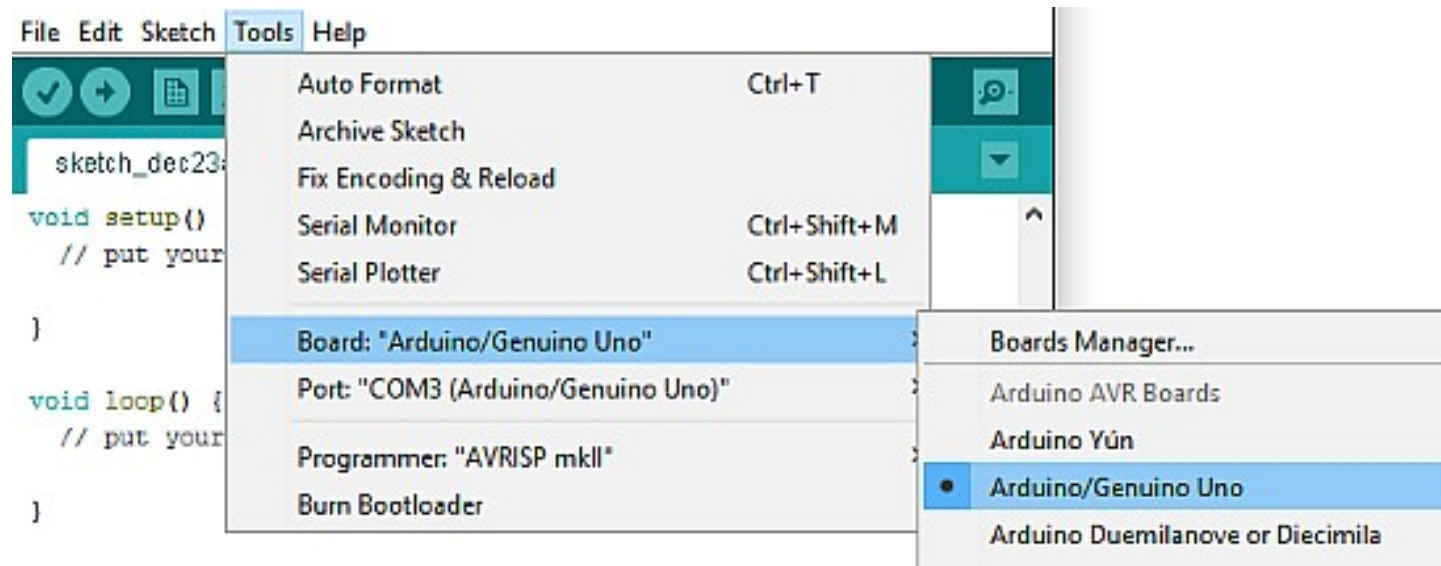


➤ Tela principal do E3 Studio



Configurando o Arduino IDE

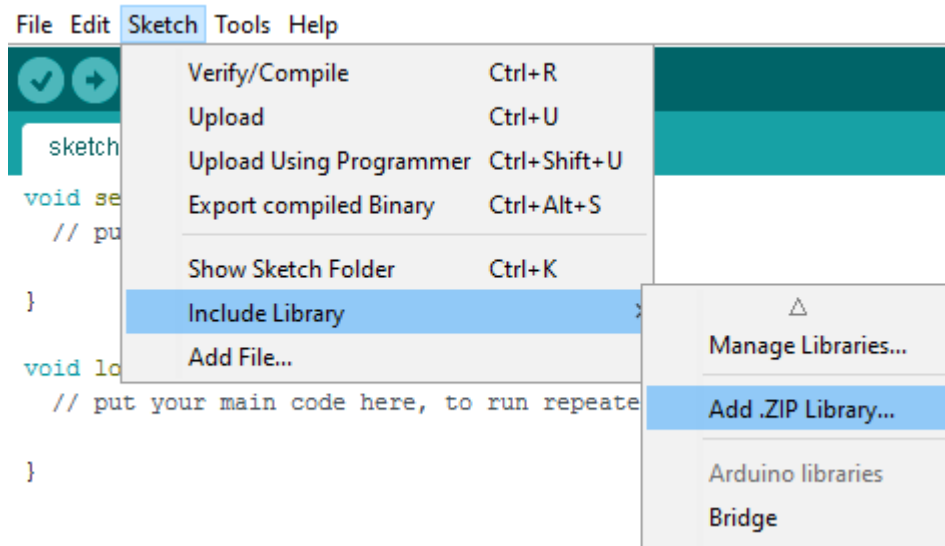
- Configuração de placa e porta de comunicação
- Configure o IDE para que ele trabalhe com a placa (board) e a porta (port) específicas do seu equipamento.



Configurando o protocolo Modbus Slave no Arduino

➤ Adicionando a biblioteca **arduino-modbus-slave**

- Após as configurações físicas é necessário configurar o hardware como uma unidade escrava para a rede Modbus. Para tal, é preciso uma biblioteca com as configurações do protocolo.
- Dentre as várias disponíveis, a arduino-modbus-slave será incluída na IDE para a efetiva comunicação.



Configurando o protocolo Modbus Slave no Arduino

➤ Carregue o script abaixo no Arduino para torna-lo um dispositivo escravo

```
#include <modbus.h>
#include <modbusDevice.h>
#include <modbusRegBank.h>
#include <modbusSlave.h>

modbusDevice regBank;// banco de reg.
modbusSlave slave;

//Define AI variables
word AI0,AI1,AI2,AI3,AI4,AI5;

void setup()
{
  //Set Slave ID
  regBank.setId(1);//endereço modbus (0
  a 127)

  //Add Digital Input registers
  regBank.add(10002);
  regBank.add(10003);
  regBank.add(10004);
  regBank.add(10005);
  regBank.add(10006);
  regBank.add(10007);
```

```
//Add Digital Output registers
regBank.add(8);
regBank.add(9);
regBank.add(12);
regBank.add(13);
```

```
//Analog Input registers
regBank.add(30001);
regBank.add(30002);
regBank.add(30003);
regBank.add(30004);
regBank.add(30005);
regBank.add(30006);
```

```
//Analog Output registers
regBank.add(40010);
regBank.add(40011);
```

```
slave._device = &regBank;
```

```
//Set BaudRate
slave.setBaud(9600);
```

```
//Configure pins
pinMode(2,INPUT); //Push button
pinMode(3,INPUT);
pinMode(4,INPUT);
pinMode(5,INPUT);
pinMode(6,INPUT);
pinMode(7,INPUT);
pinMode(8,OUTPUT); //Yellow LED
(ON/OFF one button)
pinMode(9,OUTPUT);
pinMode(12,OUTPUT); //Green LED
(ON/OFF two buttons)
pinMode(13,OUTPUT);
}
```

Configurando o protocolo Modbus Slave no Arduino

➤ Carregue o script abaixo no Arduino para torna-lo um dispositivo escravo

```
void loop()
{
  //Digital Output
  byte DO8 = regBank.get(8);
  if (DO8 <= 0 && digitalRead(8) == HIGH)digitalWrite(8,LOW);
  if (DO8 >= 1 && digitalRead(8) == LOW)digitalWrite(8,HIGH);

  byte DO9 = regBank.get(9);
  if (DO9 <= 0 && digitalRead(9) == HIGH)digitalWrite(9,LOW);
  if (DO9 >= 1 && digitalRead(9) == LOW)digitalWrite(9,HIGH);

  byte DO12 = regBank.get(12);
  if (DO12 <= 0 && digitalRead(12) == HIGH)digitalWrite(12,LOW);
  if (DO12 >= 1 && digitalRead(12) == LOW)digitalWrite(12,HIGH);

  byte DO13 = regBank.get(13);
  if (DO13 <= 0 && digitalRead(13) == HIGH)digitalWrite(13,LOW);
  if (DO13 >= 1 && digitalRead(13) == LOW)digitalWrite(13,HIGH);

  //Digital Input
  byte DI2 = digitalRead(2);
  if (DI2 >= 1)regBank.set(10002,1);
  if (DI2 <= 0)regBank.set(10002,0);
```

```
  byte DI3 = digitalRead(3);
  if (DI3 >= 1)regBank.set(10003,1);
  if (DI3 <= 0)regBank.set(10003,0);

  byte DI4 = digitalRead(4);
  if (DI4 >= 1)regBank.set(10004,1);
  if (DI4 <= 0)regBank.set(10004,0);

  byte DI5 = digitalRead(5);
  if (DI5 >= 1)regBank.set(10005,1);
  if (DI5 <= 0)regBank.set(10005,0);

  byte DI6 = digitalRead(6);
  if (DI6 >= 1 )regBank.set(10006,1);
  if (DI6 <= 0)regBank.set(10006,0);

  byte DI7 = digitalRead(7);
  if (DI7 >= 1)regBank.set(10007,1);
  if (DI7 <= 0)regBank.set(10007,0);
```

Configurando o protocolo Modbus Slave no Arduino

➤ Carregue o script abaixo no Arduino para torna-lo um dispositivo escravo

//Analog Input

AI0 = analogRead(0); //Potentiometer

delay(10);

AI0 = analogRead(0);

regBank.set(30001, (word) AI0);

delay(10);

AI1 = analogRead(1);

delay(10);

AI1 = analogRead(1);

regBank.set(30002, (word) AI1);

delay(10);

AI2 = analogRead(2);

delay(10);

AI2 = analogRead(2);

regBank.set(30003, (word) AI2);

delay(10);

AI3 = analogRead(3);

delay(10);

AI3 = analogRead(3);

regBank.set(30004, (word) AI3);

delay(10);

AI4 = analogRead(4);

delay(10);

AI4 = analogRead(4);

regBank.set(30005, (word) AI4);

delay(10);

AI5 = analogRead(5);

delay(10);

AI5 = analogRead(5);

regBank.set(30006, (word) AI5);

delay(10);

//Analog Output (PWM)

word AO10 = regBank.get(40010); //Red LED (change the brightness)

analogWrite(10,AO10);

delay(10);

word AO11 = regBank.get(40011);

analogWrite(11,AO11);

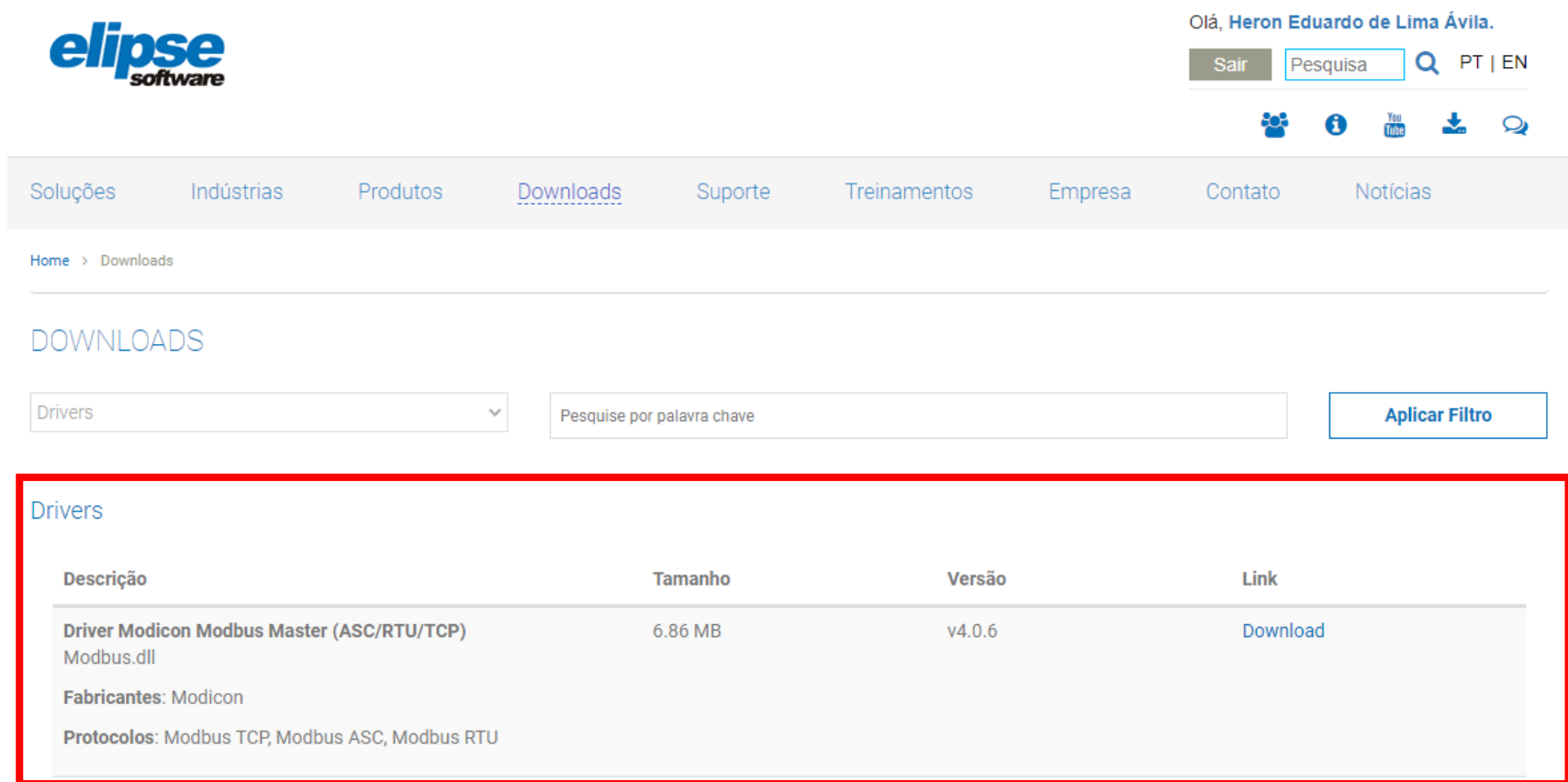
delay(10);

slave.run();

}

Configurando a comunicação com o E3 Studio

- Baixe o driver Modicon Modbus Master (ASC/RTU/TCP) no site da Elipse Software (precisa criar um usuário).



Olá, [Heron Eduardo de Lima Ávila](#).

[Sair](#) [Q](#) [PT](#) | [EN](#)

[Soluções](#) [Indústrias](#) [Produtos](#) [Downloads](#) [Suporte](#) [Treinamentos](#) [Empresa](#) [Contato](#) [Notícias](#)

[Home](#) > [Downloads](#)

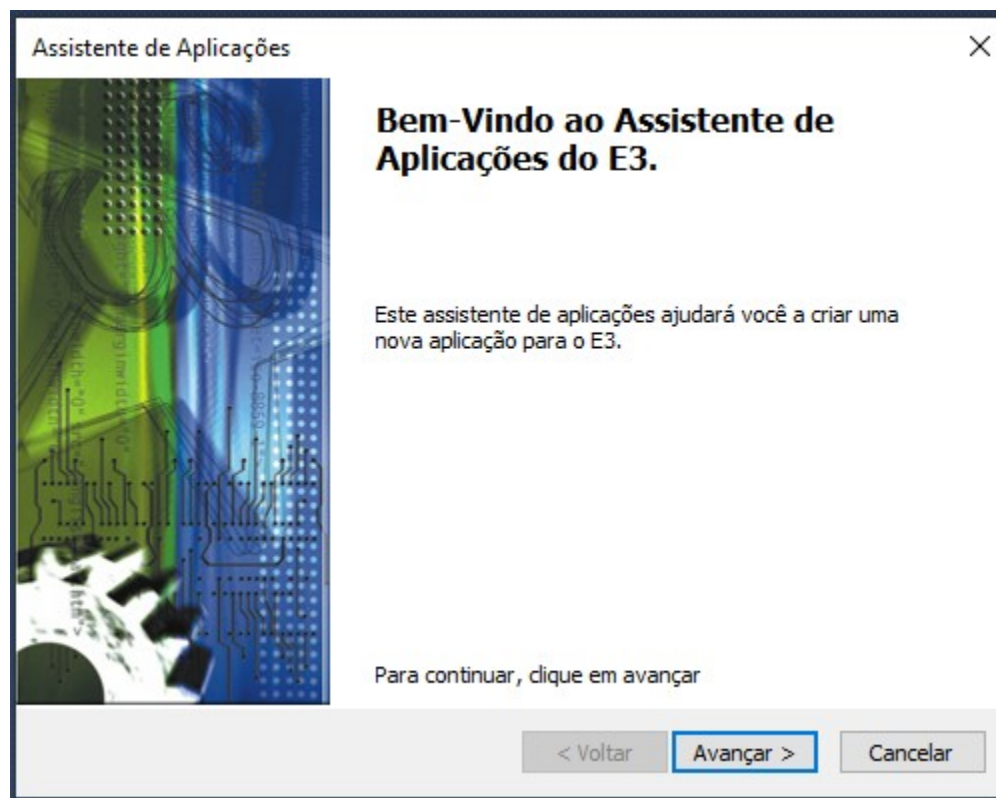
DOWNLOADS

Drivers [Aplicar Filtro](#)

Descrição	Tamanho	Versão	Link
Driver Modicon Modbus Master (ASC/RTU/TCP) Modbus.dll Fabricantes: Modicon Protocolos: Modbus TCP, Modbus ASC, Modbus RTU	6.86 MB	v4.0.6	Download

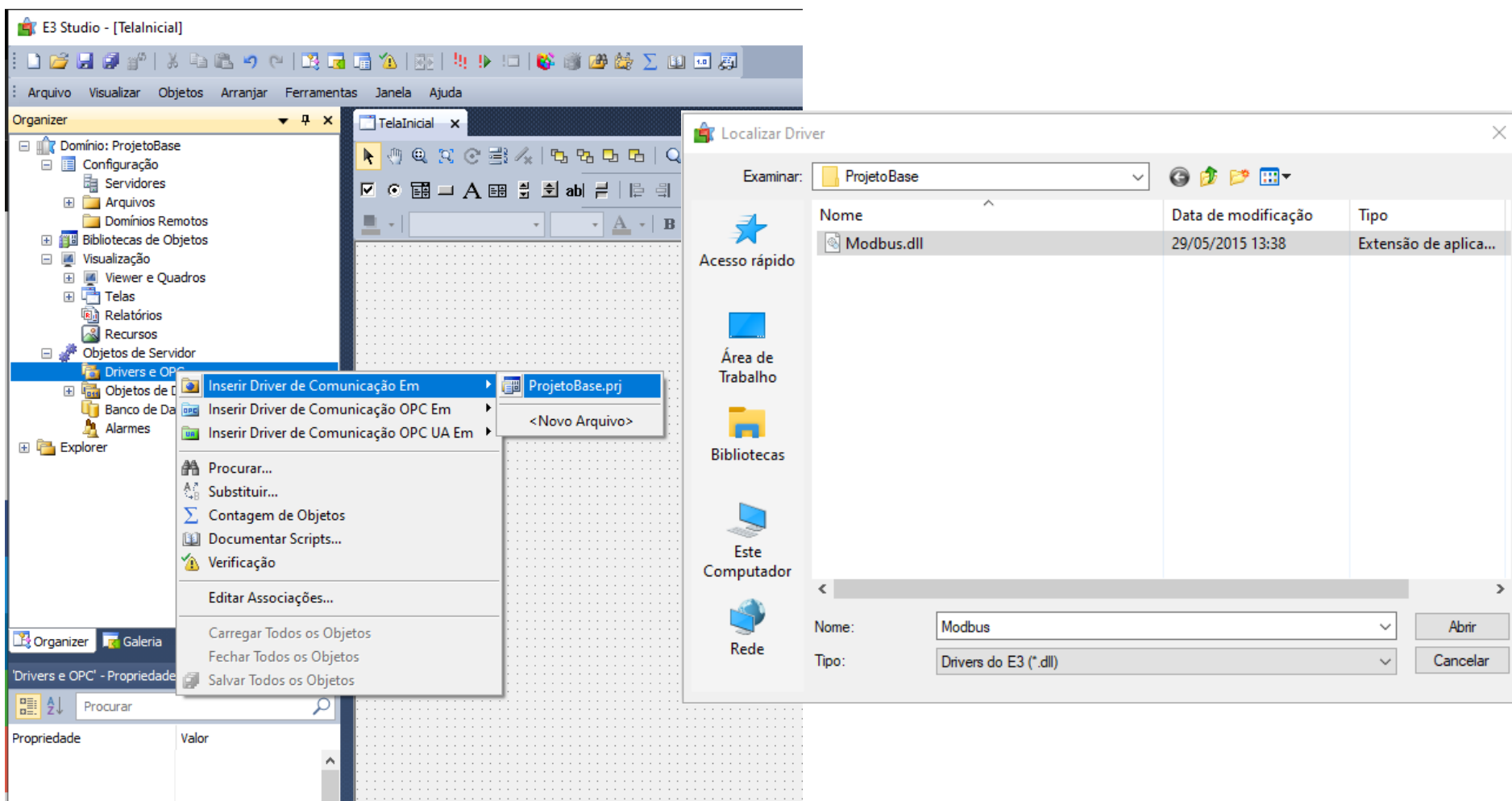
Configurando a comunicação com o E3 Studio

- Abra o E3 Studio, crie um novo projeto e configure uma nova aplicação padrão, sem ajustes específicos.



Configurando a comunicação com o E3 Studio

- Adicione o driver Modicon Modbus Master (ASC/RTU/TCP) à aplicação



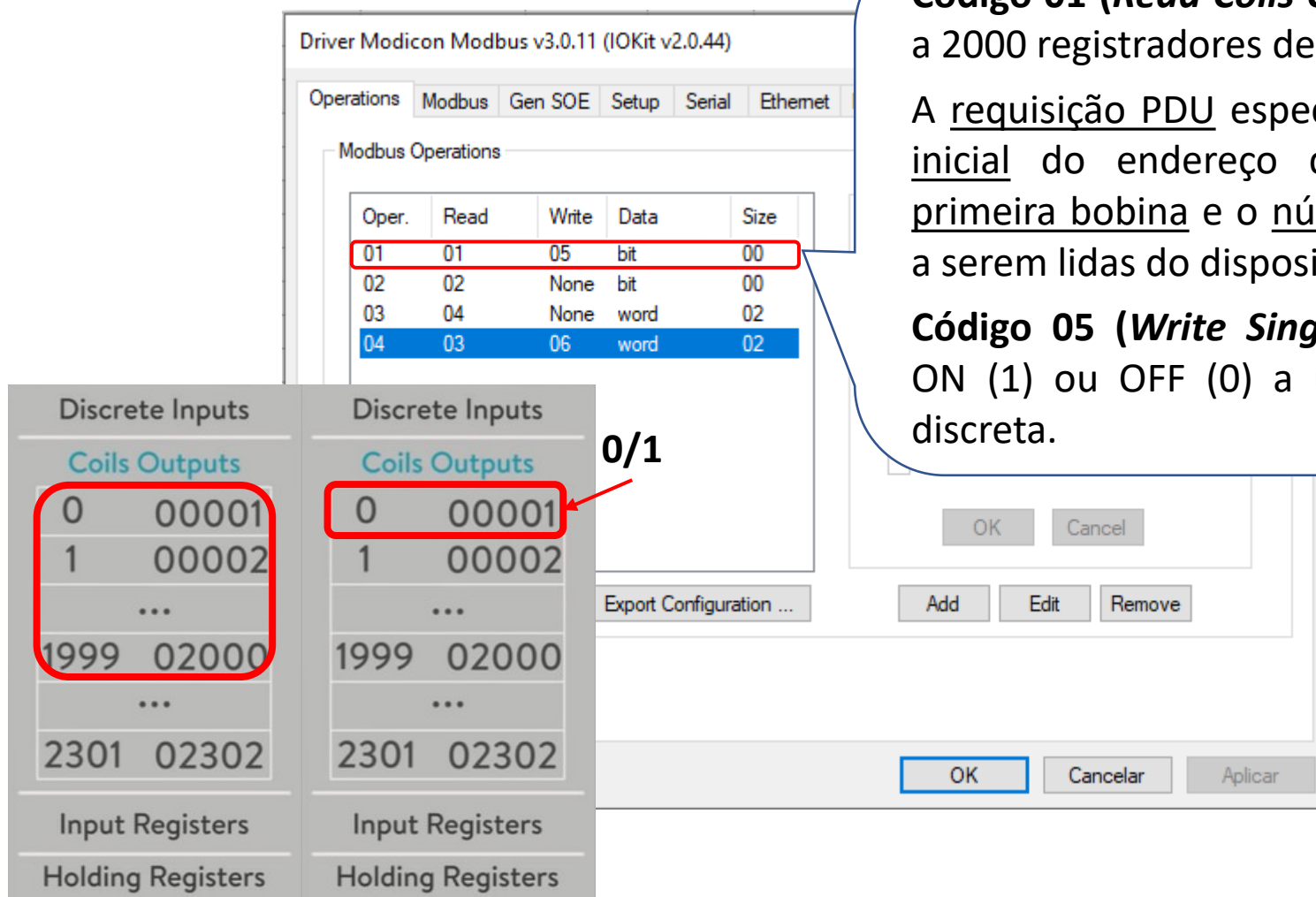
The screenshot displays the E3 Studio software interface. The 'Organizer' pane on the left shows a project tree for 'ProjetoBase'. A context menu is open over the 'Drivers e OPC' folder, with the option 'Inserir Driver de Comunicação Em' selected, leading to a submenu where 'ProjetoBase.prj' is chosen. The 'Localizar Driver' dialog box is open, showing a search for 'Modbus.dll' in the 'ProjetoBase' directory. The dialog includes a table of search results and fields for 'Nome' and 'Tipo'.

Nome	Data de modificação	Tipo
Modbus.dll	29/05/2015 13:38	Extensão de aplica...

At the bottom of the dialog, the 'Nome' field is set to 'Modbus' and the 'Tipo' field is set to 'Drivers do E3 (*.dll)'. Buttons for 'Abrir' and 'Cancelar' are visible.

Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



Driver Modicon Modbus v3.0.11 (IOKit v2.0.44)

Operations: Modbus | Gen SOE | Setup | Serial | Ethernet

Modbus Operations

Oper.	Read	Write	Data	Size
01	01	05	bit	00
02	02	None	bit	00
03	04	None	word	02
04	03	06	word	02

Discrete Inputs

Coils Outputs

0	00001
1	00002
...	
1999	02000
...	
2301	02302

Input Registers

Holding Registers

0/1

OK Cancel

Add Edit Remove

Export Configuration ...

OK Cancelar Aplicar

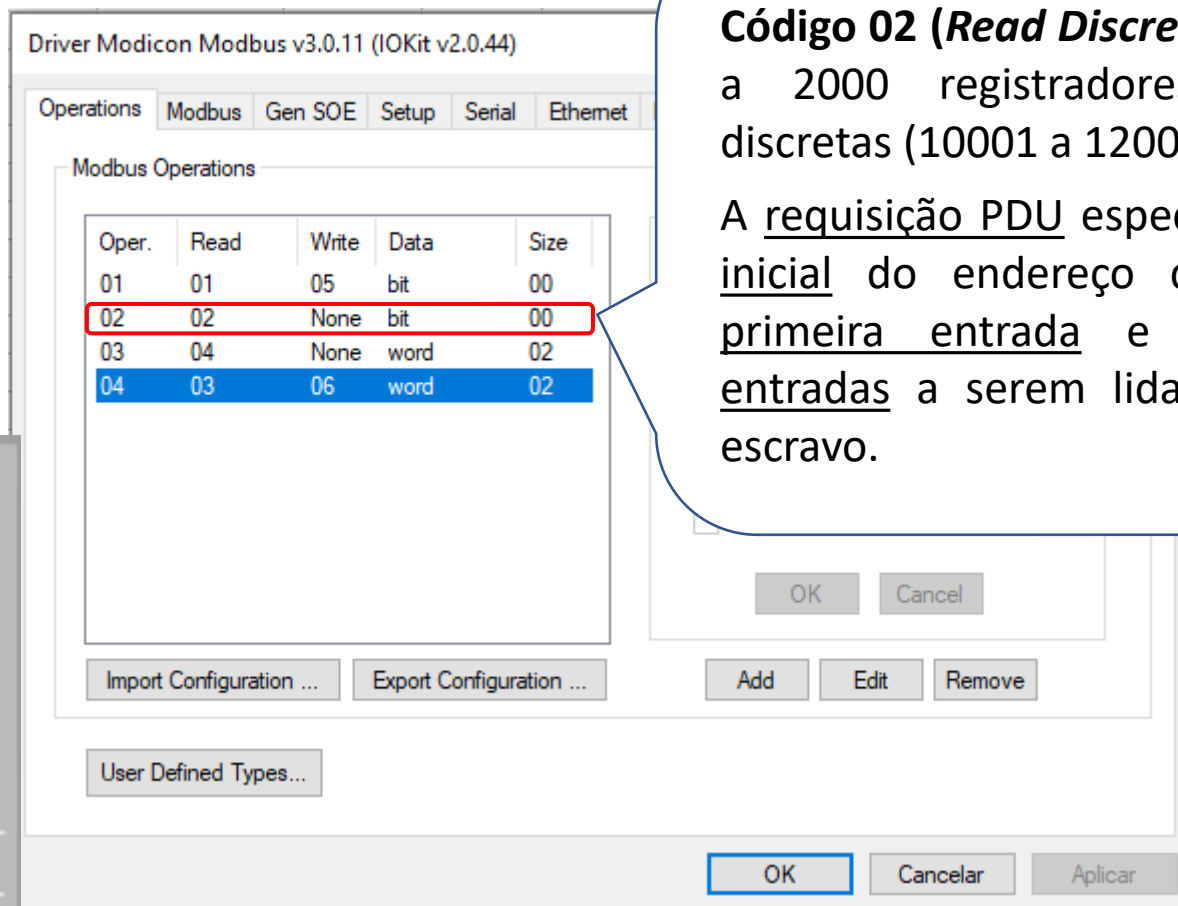
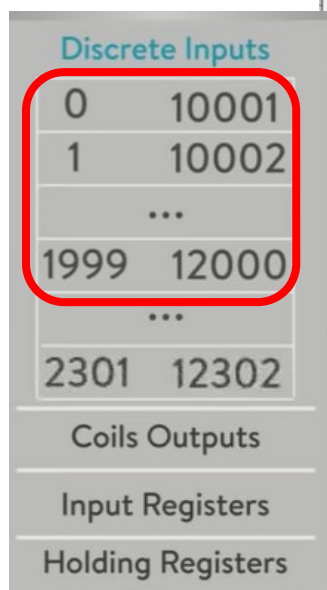
Código 01 (*Read Coils Outputs*): Lê de 1 a 2000 registradores de saídas discretas.

A requisição PDU especifica o endereço inicial do endereço de memória da primeira bobina e o número de bobinas a serem lidas do dispositivo escravo.

Código 05 (*Write Single Coil*): Escreve ON (1) ou OFF (0) a uma única saída discreta.

Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.

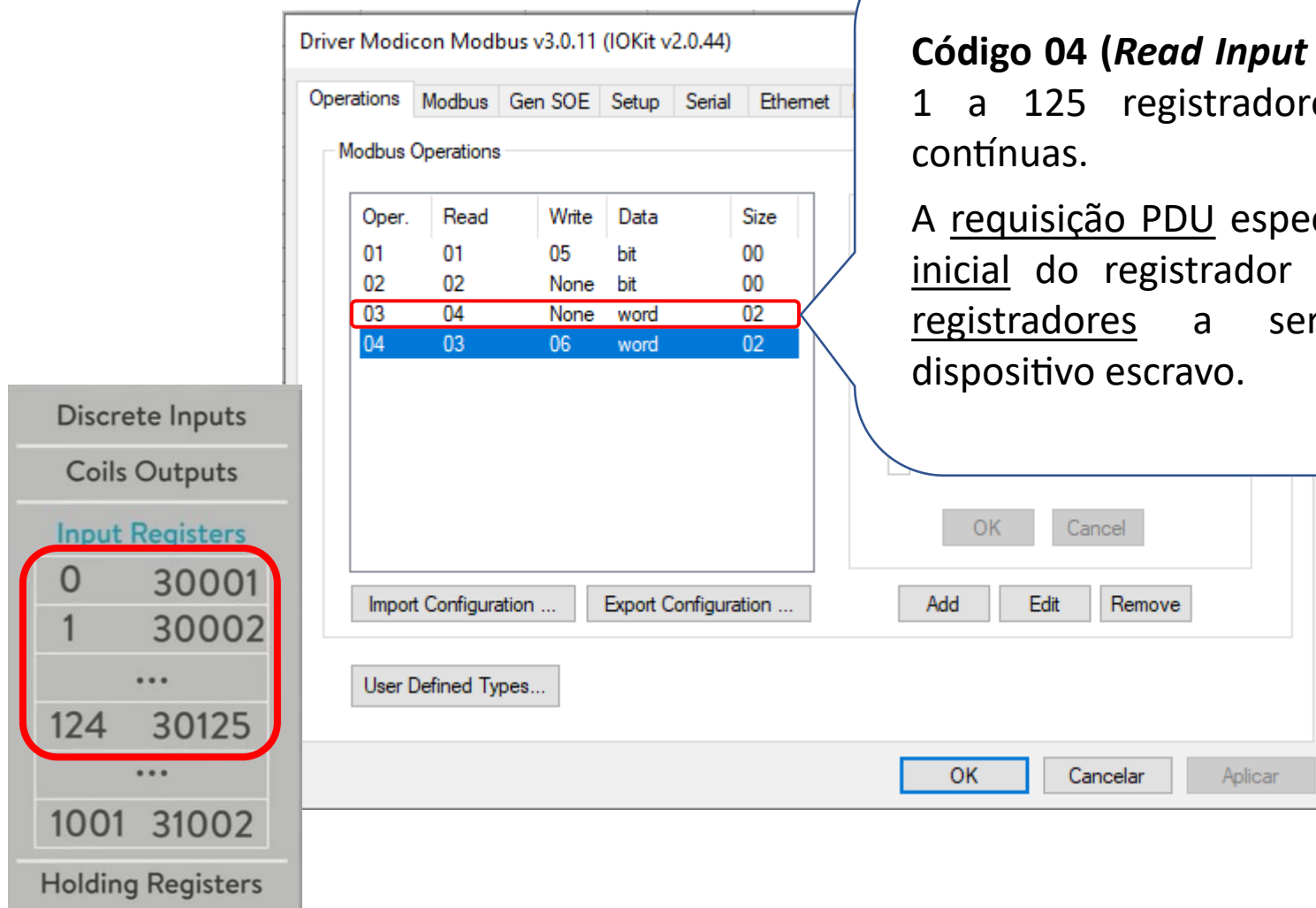


Código 02 (*Read Discret Inputs*): Lê de 1 a 2000 registradores de entradas discretas (10001 a 12000).

A requisição PDU especifica o endereço inicial do endereço de memória da primeira entrada e o número de entradas a serem lidas do dispositivo escravo.

Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.

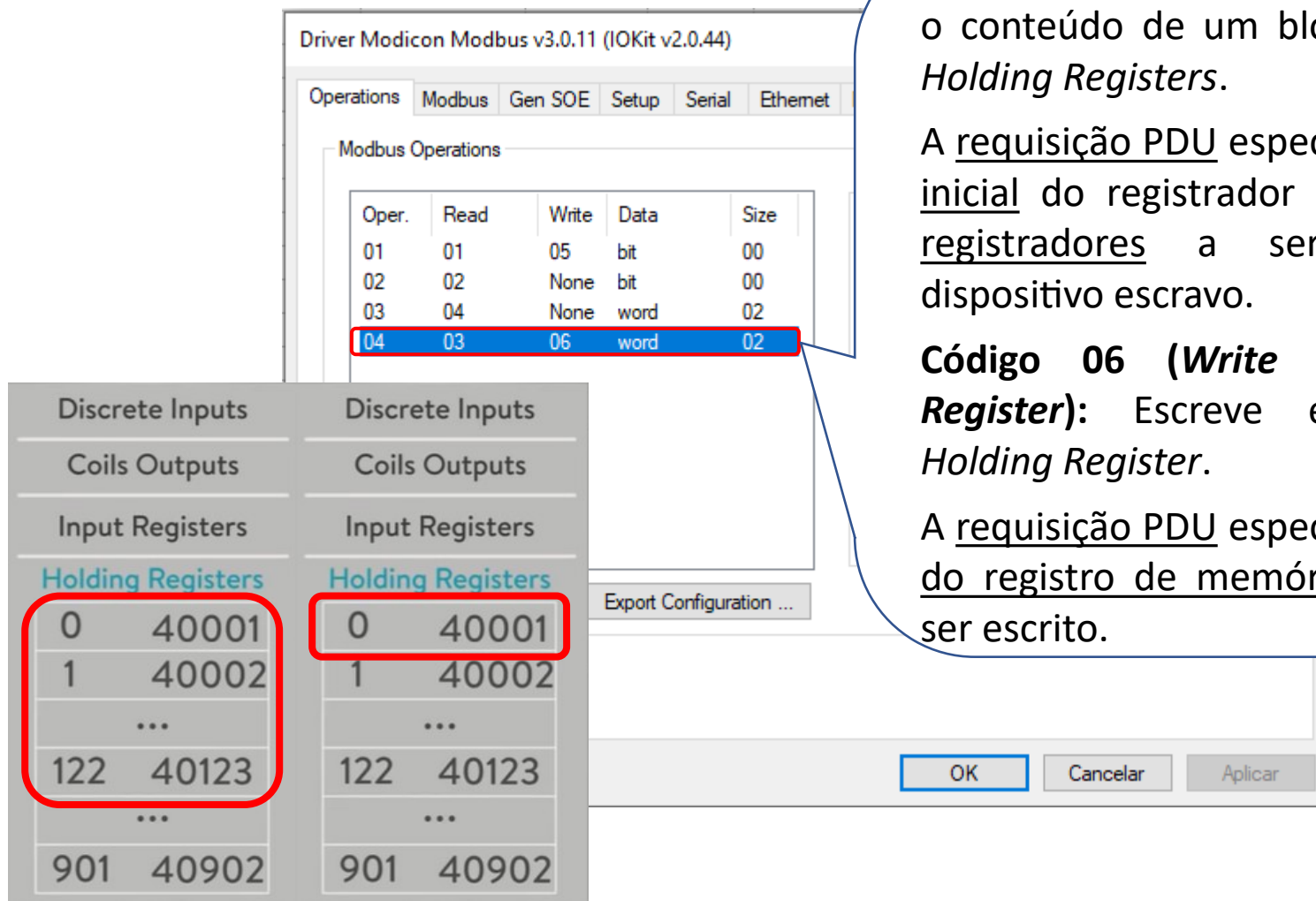


Código 04 (Read Input Registers): Lê de 1 a 125 registradores de entradas contínuas.

A requisição PDU especifica o endereço inicial do registrador e o número de registradores a serem lidos do dispositivo escravo.

Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



Driver Modicon Modbus v3.0.11 (IOKit v2.0.44)

Operations: Modbus | Gen SOE | Setup | Serial | Ethernet

Modbus Operations

Oper.	Read	Write	Data	Size
01	01	05	bit	00
02	02	None	bit	00
03	04	None	word	02
04	03	06	word	02

Export Configuration ...

OK Cancelar Aplicar

Discrete Inputs

Coils Outputs

Input Registers

Holding Registers

0	40001
1	40002
...	...
122	40123
...	...
901	40902

Código 03 (Read Holding Registers): Lê o conteúdo de um bloco contínuo de *Holding Registers*.

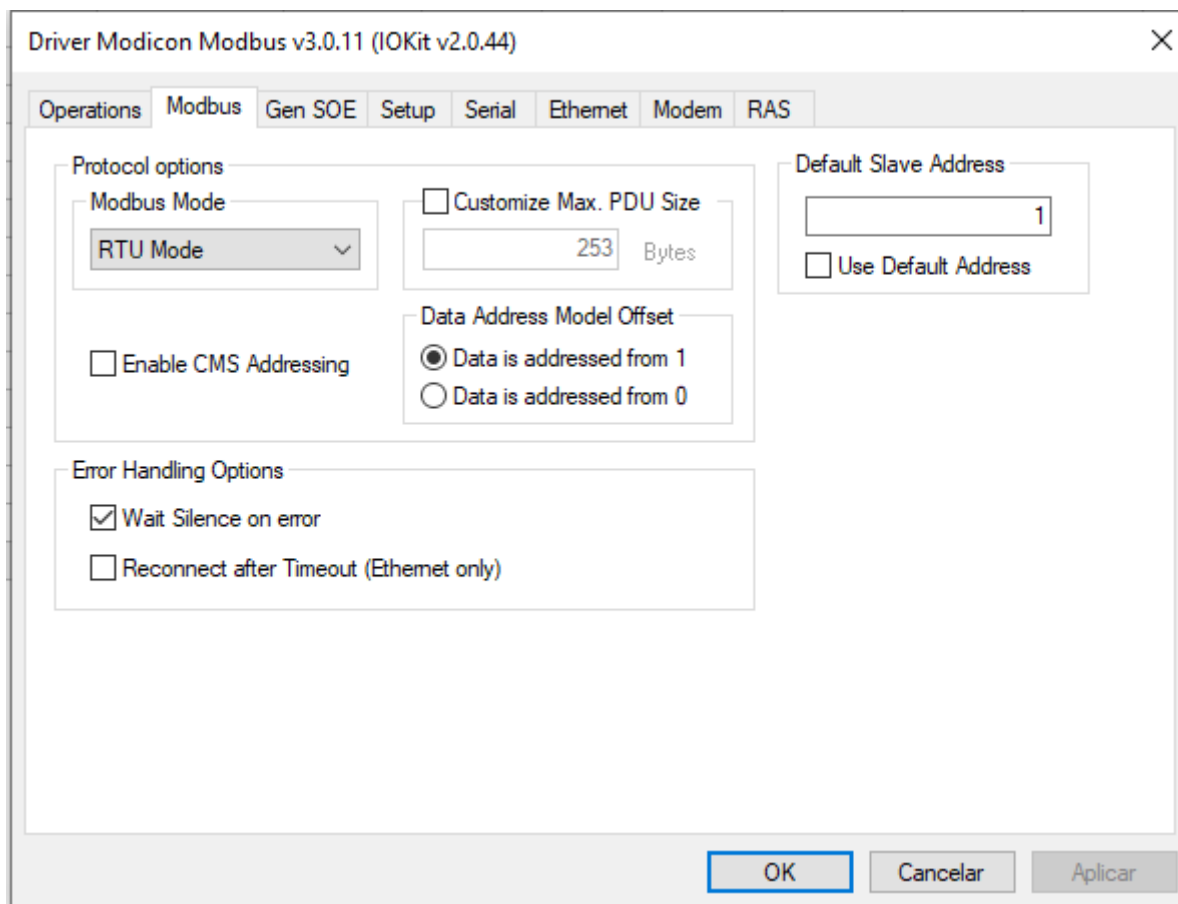
A requisição PDU especifica o endereço inicial do registrador e o número de registradores a serem lidos do dispositivo escravo.

Código 06 (Write Single Holding Register): Escreve em um único *Holding Register*.

A requisição PDU especifica o endereço do registro de memória do escravo a ser escrito.

Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



The screenshot shows the configuration window for the Driver Modicon Modbus v3.0.11 (IOKit v2.0.44). The window has a title bar with a close button (X) and a tabbed interface with the following tabs: Operations, Modbus (selected), Gen SOE, Setup, Serial, Ethernet, Modem, and RAS.

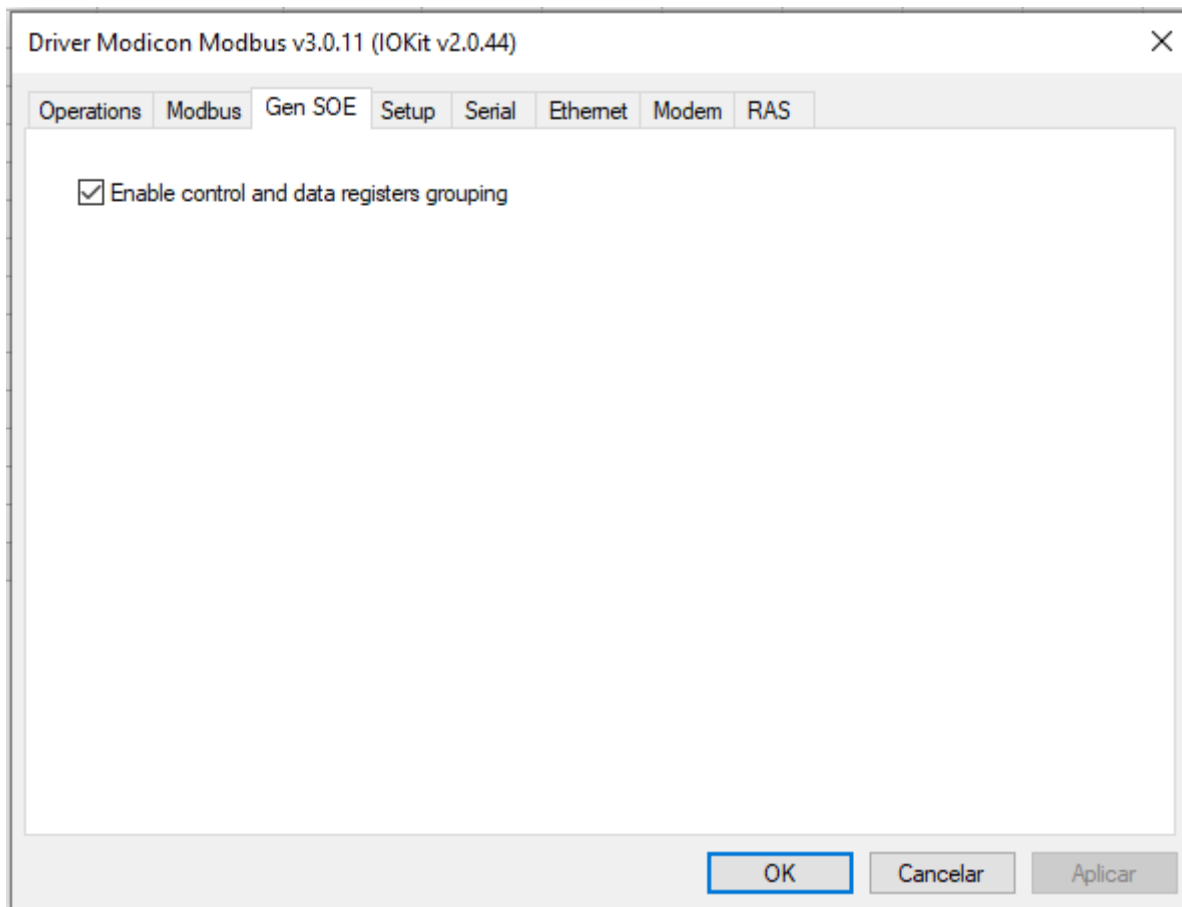
The Modbus tab contains the following settings:

- Protocol options:**
 - Modbus Mode:** A dropdown menu set to "RTU Mode".
 - ☐ Customize Max. PDU Size: A checkbox that is unchecked. Below it is a text box containing "253" followed by "Bytes".
 - ☐ Enable CMS Addressing: A checkbox that is unchecked.
 - Data Address Model Offset:**
 - ☒ Data is addressed from 1
 - ☐ Data is addressed from 0
- Default Slave Address:**
 - A text box containing "1".
 - ☐ Use Default Address: A checkbox that is unchecked.
- Error Handling Options:**
 - ☒ Wait Silence on error
 - ☐ Reconnect after Timeout (Ethernet only)

At the bottom of the window, there are three buttons: "OK" (highlighted with a blue border), "Cancelar", and "Aplicar".

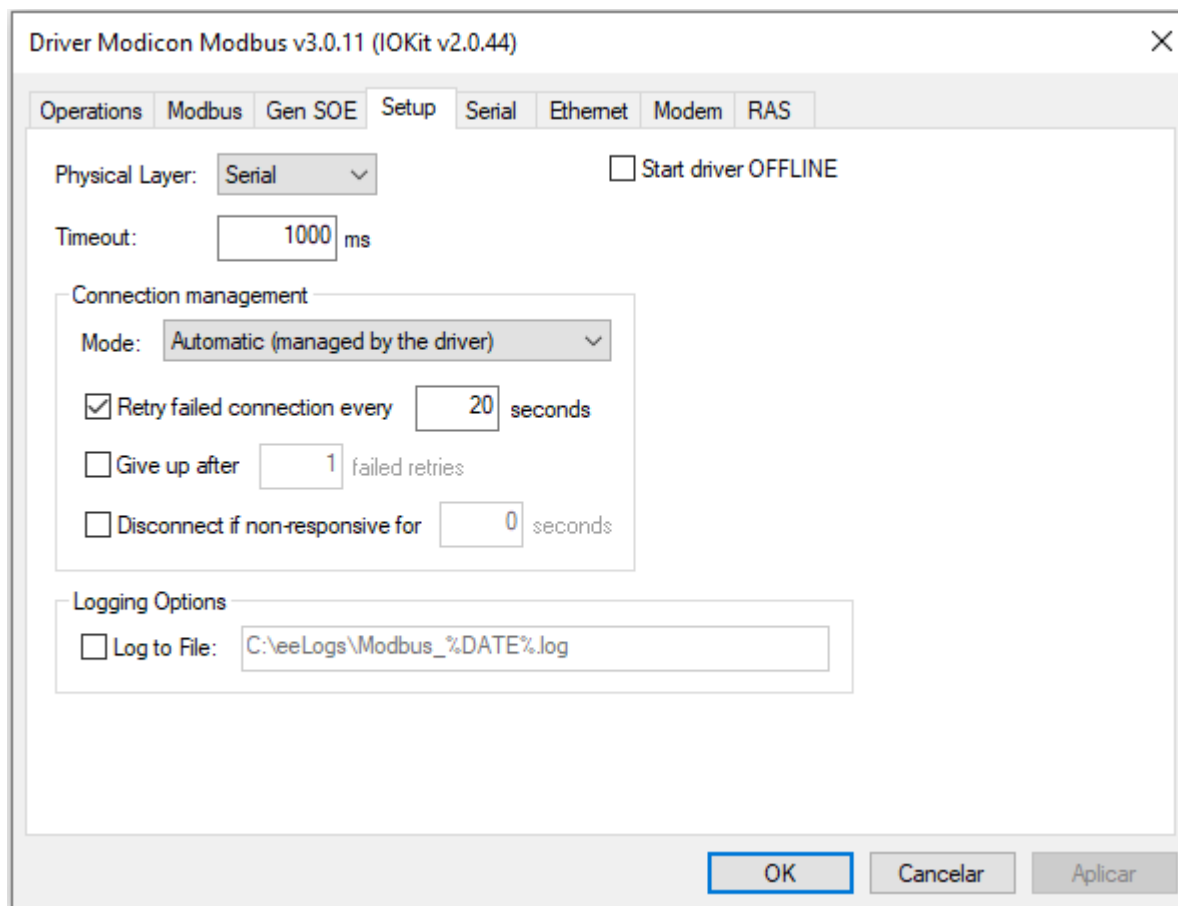
Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



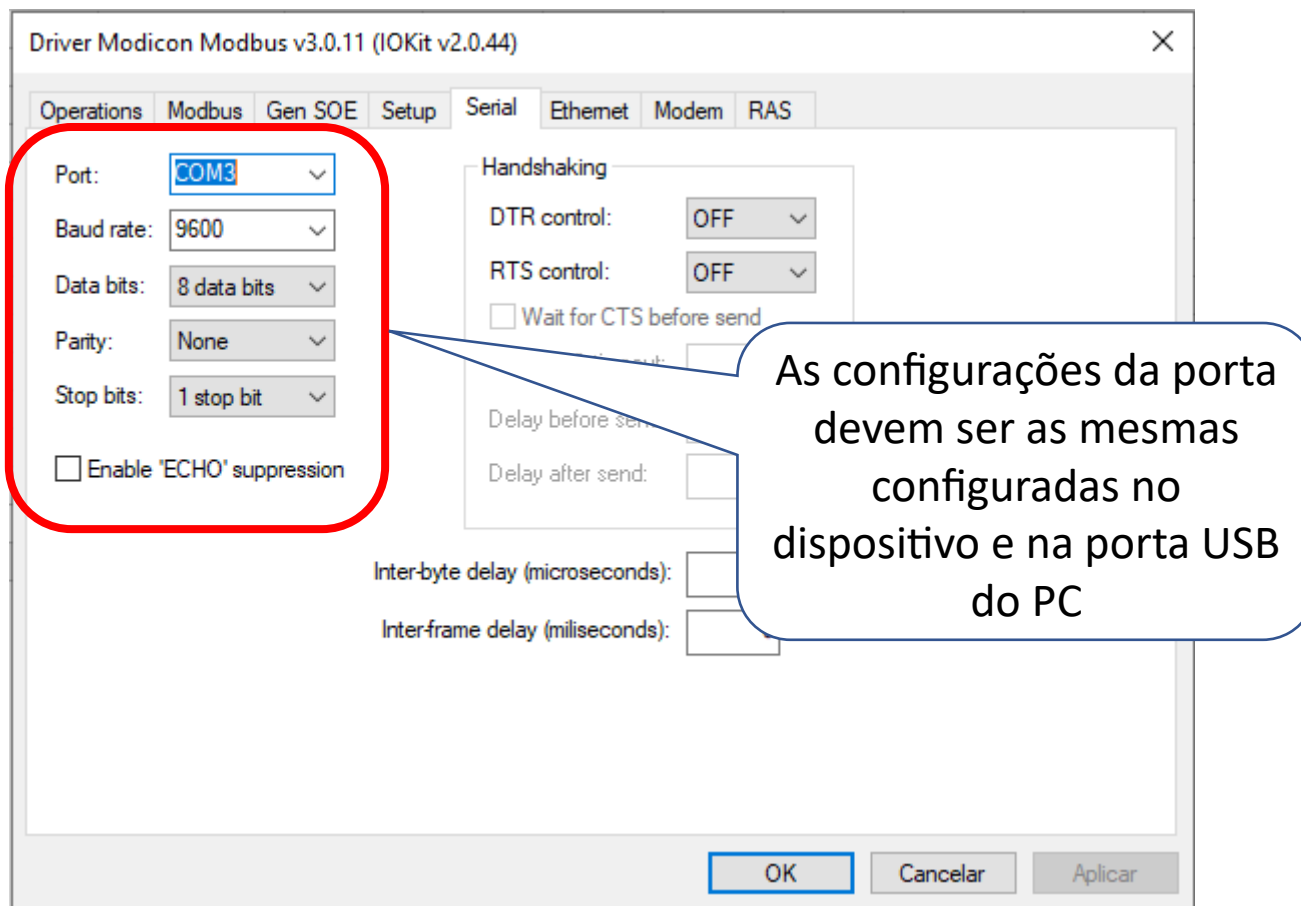
Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



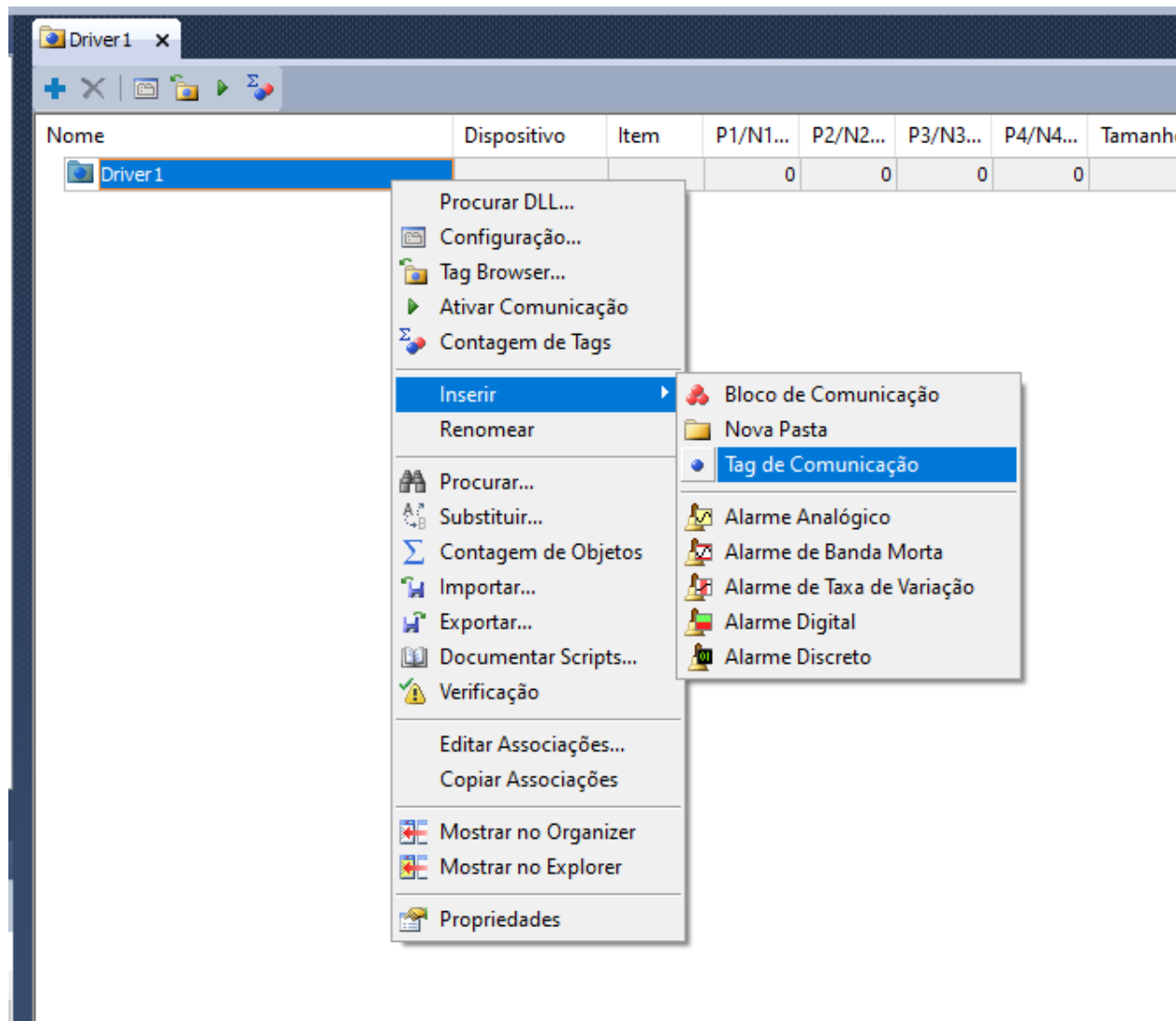
Configurando o driver Modicon Modbus Master (ASC/RTU/TCP)

- Acesse as configurações do driver (IOKit) e parametrize-as conforme as figuras abaixo.



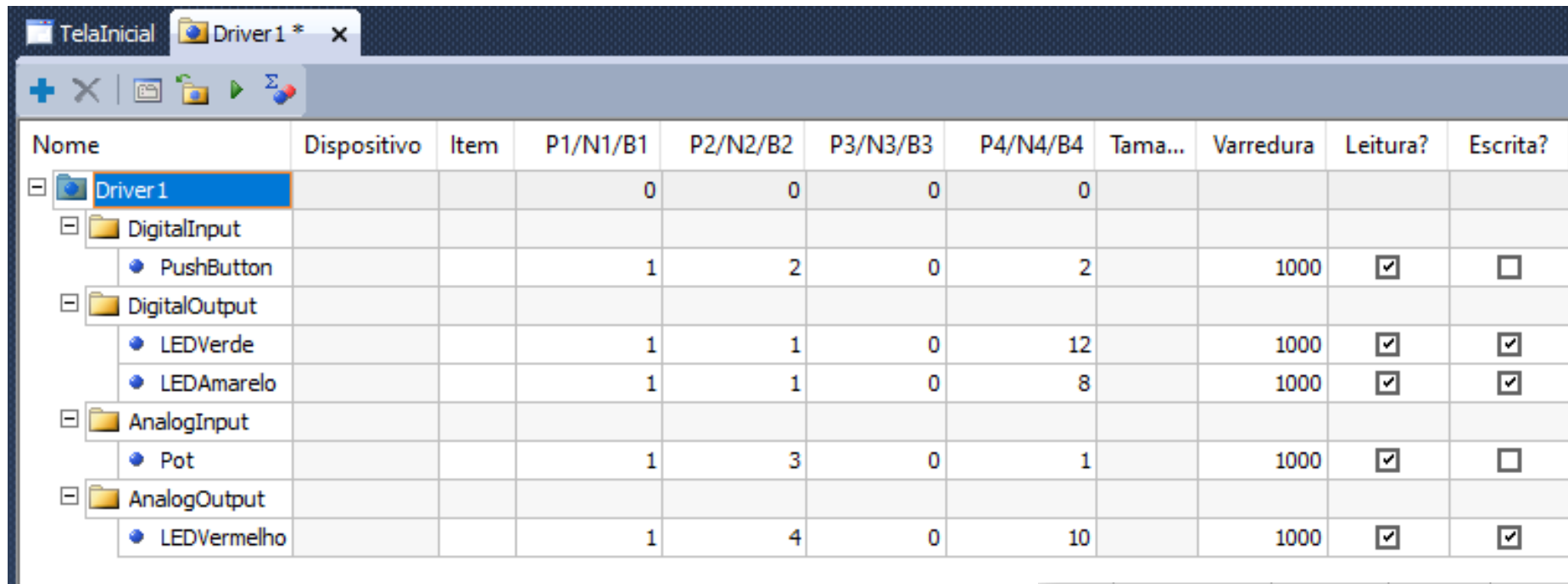
Configurando as Tags de comunicação do driver

- Insira as Tags necessárias para a aplicação



Configurando as Tags de comunicação do driver

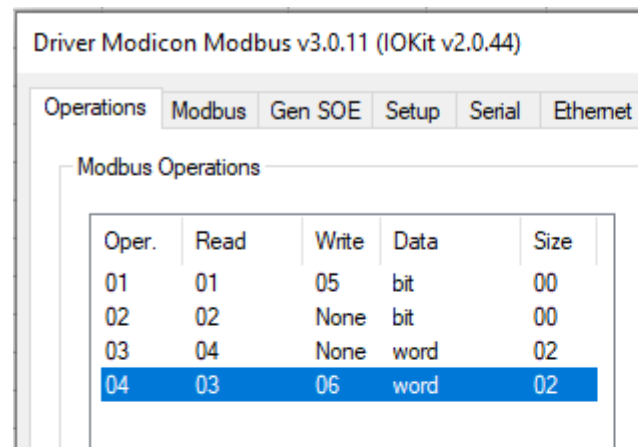
➤ Insira as Tags necessárias para a aplicação



Nome	Dispositivo	Item	P1/N1/B1	P2/N2/B2	P3/N3/B3	P4/N4/B4	Tama...	Varredura	Leitura?	Escrita?
Driver1			0	0	0	0				
DigitalInput										
PushButton			1	2	0	2		1000	☒	☐
DigitalOutput										
LEDVerde			1	1	0	12		1000	☒	☒
LEDAmarelo			1	1	0	8		1000	☒	☒
AnalogInput										
Pot			1	3	0	1		1000	☒	☐
AnalogOutput										
LEDVermelho			1	4	0	10		1000	☒	☒

- **N1:** Endereço do equipamento escravo (Arduino).
- **N2:** Código da operação (conforme configurado no IOKit).
- **N3:** Parâmetro adicional, não utilizado (deixar em 0).
- **N4:** Endereço do registro do Arduino a ser lido/escrito.

01: Read coils outputs / **05:** Write single coil / **02:** Read discrete inputs
04: Read input registers / **03:** Read holding registers / **06:** Write single holding register



Driver Modicon Modbus v3.0.11 (IOKit v2.0.44)

Operations Modbus Gen SOE Setup Serial Ethernet

Modbus Operations

Oper.	Read	Write	Data	Size
01	01	05	bit	00
02	02	None	bit	00
03	04	None	word	02
04	03	06	word	02

Referências

1. FREIRE, Luiz Fernando. **Comunicando o Arduino com Elipse E3 e/ou Elipse SCADA**. Elipse Knowledgebase, 2019. Disponível em: <<https://kb.elipse.com.br/comunicando-o-arduino-com-elipse-e3-eou-elipse-scada/>>. Acesso em: 01 dez. 2021.
2. MODBUS Organization. **MODBUS Application Protocol Specification**. Modbus Specifications and Implementation Guides, V1.1b3, 2012. 50 p.
3. RealPars. **What is Modbus and How does it Work?** Youtube. Disponível em: <https://www.youtube.com/watch?v=txi2p5_OjKU&ab_channel=RealPars>. Acesso em: 19 nov. 2021.